No.

Date

cloud：云技术，按需提供资源

service model：IaaS（Infrastructure），基础，如：创建/管理一台虚拟机

FaaS（Function） 函数，如：生命周期很短功能

PaaS（Platform） 平台，如：上传资源至平台

SaaS（Software） 软件，如：在线应用（帐号）

五大特征：On-demand self-service：按需自助

Broad network access：广泛网络访问

Resource pooling：资源池化

Rapid elasticity：快速弹性收缩

Measured service：可计量服务

故全天候运行不算cloud.

Capical expenditure：CapEx 资本性支出

Operational expenditure：OpEx 运营性支出

扫描全能王 创建

No.

Date

Work：工作目标，全部运算操作的总量

使用下衡量：仅用1个 worker 所用时间

Task：任务，计算分片，将总工作切分得更小的片段

Task之间可以存在依赖关系

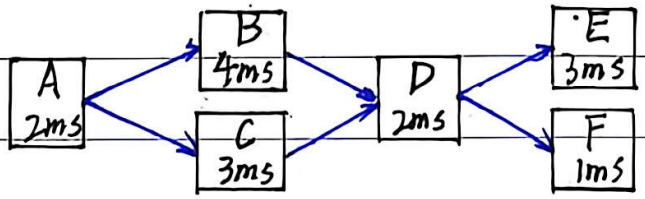
Worker：执行单元，实际执行运算的算力载体

指取 tasks 并执行

dependency graph and minimum completion time：

关键路径　最小完成时间

Task	Start	Finish
A	0	2
B	2	6
C	2	5
D	6	8
E	8	11
F	8	9



critical path：A → B → D → E

2 + 4 + 2 + 3 = 11ms

Data Parallelism：数据并行　Task Parallelism：任务并行

x_0 x_1 x_2 x_3 x_4 x_5 ... x_n 同样操作

↓ ↓ ↓ ↓ ↓ ↓ ↓ 多次执行

y_0 y_1 y_2 y_3 y_4 y_5 y_n

A → B → D → E 不同操作 依次执行

Amdahl's Law：P个 workers 下的最大加速比。

$S_A(P) = \frac{1}{f + \frac{1-f}{P}}$

f：串行部分的占比　注：f为性能瓶颈，故

P：并行节点的数量　P越大，增益越小

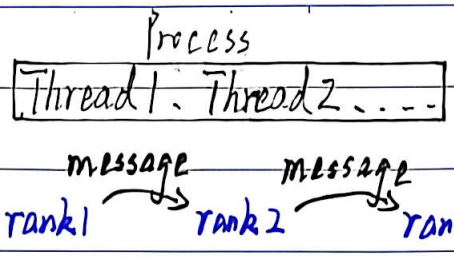
当 workers $P \to \infty$，$S \leq \frac{1}{f}$，absolute speedup limit

扫描全能王 创建

No.

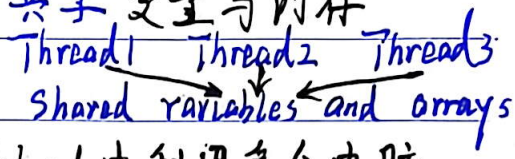
Date

属于 进程 Process：正在运行的程序
线程 Thread：进程内部执行单元



MPI：在不同节点间搬运数据，将数据拆分为多个节点各个进程

OpenMP：在单节点内利用全部 CPU 共享变量与内存



集群 SSI：单系统映像，在 cluster workload 中利用多台电脑物理上连接、以软件将集群统一为完整系统。然而，其仍会受硬件故障、传输损耗等硬件方面的故障.

cluster 集群 → node 一台计算机 → socket CPU 插槽 → core 核心、基础单元

Head node: 头节点 用户接入，提交 Tasks
Compute nodes: 计算节点 执行 Tasks
Interconnect: 互联网络 传输控制消息与程序数据
Storage: 存储系统 提供统一的文件与数据访问
I/O nodes: I/O 节点 在节点间搬运资源.

$$T_{total} \approx \frac{T_{parallelwork}}{p} + T_{serial} + T_{communication} + T_{imbalance}$$

HPC：高性能、倾尽集群算力解决复杂问题
HTC：高通量、完成大量独立 Tasks
HA：高可用、发生硬件故障后仍可持续监视平台

扫描全能王 创建

No.

Date

When does a job fit: 申请CPU ≤ 节点空闲CPU
申请内存 ≤ 节点空闲内存

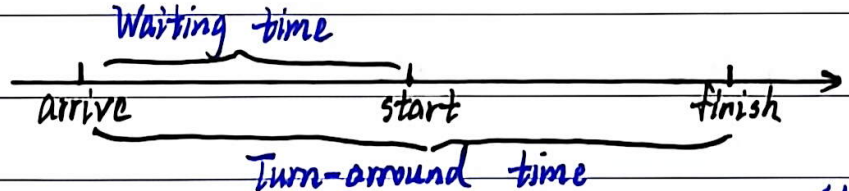
job scheduler: 如 slurm，工作负载管理器：
为用户分配一段计算时间内的计算资源 否则：资源调配失衡
在已分配的资源上启动监控任务.
根据申请量严格控制资源解决多作业竞争 计算资源被个别用户占用

CPU allocated = tasks × CPUs per task
分配CPU总数 任务数 每个任务的CPU数

core-hours = CPU allocated × hours
核时

$$\text{efficiency} = \frac{\text{core-hours actually used}}{\text{core-hours allocated}}$$
实际所用CPUs / 分配CPU总数

throughput：吞吐率，即单位时间内能处理多少 Tasks



$$\text{Utilisation} = \frac{\text{node-minutes used by jobs}}{\text{nodes} \times \text{total minutes}}$$
所有运行节点时长 / 节点总时长

扫描全能王 创建

No.

Date

Pareto optimality：帕累尔最优，有两个任务X、Y，X在平均等待时间、平均周转时间、资源利用率上均不逊于Y且至少有一点优于Y，则 X dominates Y

例: Job	Nodes	Runtime
A	4	30
B	2	10
C	2	10

解:	Wait:	avg.W	Turnarrunds	avg.T	Utilisation
Policy 1:	0、30、30	20	30、40、40	36.7	100%
Policy 2:	0、0、10	3.3	10、10、40	20	100%

第一任务即刻执行

若为牛栈驱动，则哪个更好：
1、A先、BC后； 2、BC先、A后

Back filling：回填调度，在不影响排队作业的前提下利用资源：
所需 nodes < 空闲 nodes　　当前时刻 + 运行时刻 ≤ 排队时刻

等待时长　公平份额　作业规模
$Priority = W_{age} \times Age + W_{fs} \times FairShare + W_{size} \times Size$
根据权重分配，越高越好.

$Fairshare = 2^{-usage/share}$

usage：用户消耗份额
share：分配到的份额

Dominant Resource Fairness (DRF)：主导资源公平性
多资源下取用户占比最高的一类比例进行对比

Energy：$E = P \times t$，所以运行速度并非越快越好

扫描全能王 创建

No.

Date

MPI: Message Passing Interface，消息传递接口，通过消息交换数据，遵循 SPMD 单程序、多数据.

例：
```
#include <mpi.h>        MPI库头文件
#include <stdio.h>

int main(int argc, char ** argv)
{
    MPI_Init(&argc, &argv);                    MPI初始化、启动环境
    int rank, size;                            全局通信子，代表启动的全部 MPI 进程
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);      获取当前 rank
    MPI_Comm_rank(MPI_COMM_WORLD, &size);      获取总进程数量
    printf("Hello from rank %d of %d \n", rank, size);
    MPI_Finalize();                            每个MPI进程均会打印一次
    return 0;
}
```
整个MPI进程

编译命令：mpicc main.c -o main　运行命令：mpirun -n X/main　（X：目标进程数，如4）

输出：
```
Hello from rank 2 of 4
Hello from rank 0 of 4
Hello from rank 3 of 4
Hello from rank 1 of 4
```
由于并行计算 ⇐ 输出顺序完全随机

创建ID：0~3（4个）

补：MPI_Barrier：计时前同步所有进程

MPI_Wtime：返回墙钟（wall-clock）时间

MPI_MAX：获取最慢进程的运行时间.

扫描全能王 创建

No.

Date

Blocking 阻塞进程:

MPI_Send / MPI_Recv 均需等本地操作完成后才返回. (参数多一个 & status)

若二者均在等待, 就触发了死锁 (deadlock)

所以可用非阻塞函数: 函数将立即返回

MPI_Isend / Irecv (&value, 1, MPI_INT, 0, 10, MPI_COMM, &request)

(value: 接收/发送缓冲区, 存入这个变量; 1: 元素个数; MPI_INT: 数据类型; 0: 接收/发送对象进程; 10: 标签, 收发必须一致; request: 按取信息源进程)

例: int right = (rank + 1) % size

int left = (rank + size - 1) % size

MPI_Request req[2];

MPI_Isend(halo_out, n, MPI_DOUBLE, right, 0, MPI_, &request[0]);

MPI_Irecv(halo_in, n, MPI_DOUBLE, left, 0, MPI_, &request[1]);

update_interior(); => 网格内部单元计算.

MPI_Waitall(2, req, MPI_STATUSES_IGNORE); 等待, 直至所有 req 完成

update_boundary(halo_in);

取上一个

作用: 相临 thread 的边界来更新下一个 thread 的边界

扫描全能王 创建

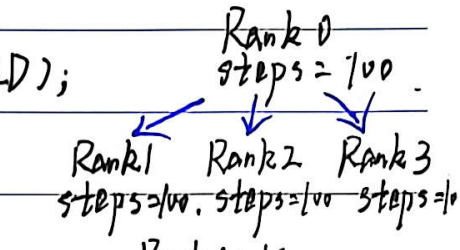
No.

Date

Collective communication：集合通信，一个函数调用多个进程

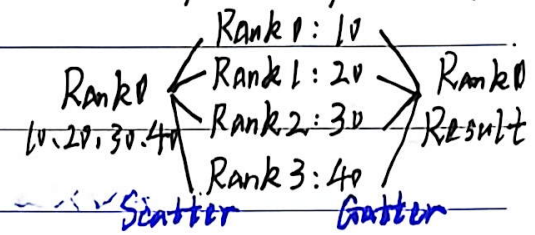
MPI_Bcast(&step, 1, MPI_INT, 0, MPI_COMM_WORLD);

将同一个变量广播至所有进程



MPI_

Scatter/Gatter：适用于广播多个不同数据



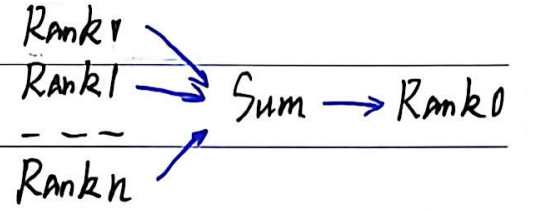
例：
```
int sendbuf[4] = {10, 20, 30, 40};
int recvbuf;
int result[4];
MPI_Scatter(sendbuf, 1, MPI_INT, &recvbuf, 1, MPI_INT, 0, MPI_COM
MPI_Gatter(&recvbuf, 1, MPI_INT, result, 1, MPI_INT, 0, MPI_Com
```

派进程　通信子

发送方的元素　　接收方的元素

MPI_Reduce：归约运算，将多个进程的结果处理并保存在 root(根进程)

例：
```
double local = 0.0;
for(long i = rank; i < n; i += size)
{
    ——— 运算
}
double total = 0.0;
MPI_Reduce(&local, &total, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COM
```



每个进程元素是　　保存在哪个进程

发送/接收缓冲区　　对元素进行什么操作

通信时近似：$T \approx \alpha + \frac{n}{\beta}$

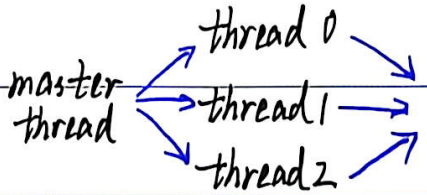
α = Latency　　β = bandwidth

扫描全能王 创建

No.

Date

OpenMP 是其一种



Fork-Join Parallelism：分叉-合并，复用 threads 降低开销.

普通例：
```
#include<omp.h>
#pragma omp parallel ← 创建 Fork-Join
{                    ← 每个 Fork 独立跑，时间可能不同，与MPI类似
  do_work[omp_get_thread_num()];
}                    ← Join 屏障，多个 Fork 全结束后才可继续
```
普通的 Fork-Join 仅对应 threads 合并，对数据无处理.

Race condition：竞态条件，在 Fork 进行写入时可能会相互覆盖.

解决：Reduction：归约，开销最低，适用于合并变量

例：
```
int total = 0;
#pragma omp parallel for reduction(+:total)
{
  total += values[i];
}
```

atomic：原子，只保护一条语句，适用于单变量或散盖.

例：
```
int total = 0;
#pragma omp parallel for shared(total, arr, n)
for(int i=0; i<n; i++)
{
  #pragma omp atomic;
  total += arr[i]; ⇒ 仅保护这一行.
}
```

critial：临界区，同一时间只允许一个 thread 进入，只适用于互斥语句，代码容易退化为串行.

omp_lock_t：手动锁，需在代码中增加 omp_init_lock/omp_destroy_lock、omp_set_lock/omp_unset_lock 来加/解锁，否则报错.

扫描全能王 创建

No.

Date

Data scope:

shared: 所有threads共享的变量，如：a[100]

private: 每个thread拥有独立副本值，如：循环部分的i

firstprivate: 最开始的初始化副本值，如：代码开头：int a=1;

lastprivate: 并行结束时最后的副本变量值.

reduction: 每个thread独立执行，最后归约，如：total

OpenMP task + depend：细分并行任务

例：
```
#pragma omp parallel
#pragma omp single   仅一个主thread创建任务.
{
  #pragma omp task depend(out:a)
  load_level(a);
  #pragma omp task depend(in:a) depend(out:b)   理解：输入项 输出项
  build_navigation(a,b);
  #pragma omp task depend(in:a) depend(out:c)
  build_visibility(a,c);
  #pragma omp task depend(in:b,c)
  publish_level(b,c);
}
```

1. load level
2. build nav
2. build visibility
3. publish

扫描全能王 创建

No.

Date

Measuring Parallel Performance：并行性能度量

$T(P,N)$：Runtime，运行时间。 P：核心数量 N：问题规模

$S(P,N) := \frac{T(1,N)}{T(P,N)}$ → 单核串行耗时 / → P核并行耗时

Speedup，加速比，判断变快了多少倍。

$E(P,N) = \frac{S(P,N)}{P}$，并行效率 Efficiency

$C(P,N) = P \cdot T(P,N)$

Parallel cost，并行开销，代表整个程序算力开销。

Amdahl's Law：计算固定负载（N不变）下的 speedup。

$S_A(P) = \frac{1}{S + \frac{1-S}{P}}$ S：串行占比 → $1-S$：并行占比

strong scaling：强缩放，效率持续下降

代表P越多，增益反而越来越少

二者差异大的原因。

Gustafson's Law：计算缩放负载下的 speedup。

$S_G(P) = S + P(1-S)$

weak scaling：弱缩放

$N(P) = P \cdot N_1$ $E = \frac{T(N_1)}{T(P \cdot N)}$

扫描全能王 创建

No.

Date

Work Law：工作量定律.

Work and span：工作量下界，标记执行时间的边界.

$T_P \geq \max(\frac{T_1}{P}, T_\infty)$ 取最大值

T_1：total work，所有任务依次串行的时间.

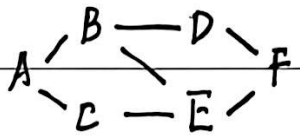
T_∞：critial/span path length，当有∞个P(核心)时的执行时间，必要时间

Parallel Parallelism：并行平均度 $A = \frac{T_1}{T_\infty}$

理论下最大同时运行的任务数

Parallel Slackness：并行松驰度 $\sigma = \frac{A}{P}$

$\sigma > 1$：任务足够；　$\sigma < 1$：核心闲置.

例： 

A：2ms、B：6ms、C：4ms

D：3ms　E：5ms　F：2ms

解：$T_1 = 2+6+4+3+5+2\text{ms} = 22\text{ms}$

$T_\infty = 2+6+5+2\text{ms} = 15\text{ms}$

$\therefore A = \frac{T_1}{T_\infty} = \frac{22}{15}$　$\frac{T_1}{P} = \frac{22}{4} = 5.5\text{ms}$

$T_P \geq \max(\frac{T_1}{P}, T_\infty) = \max(5.5, 15) = 15\text{m}$

扫描全能王 创建

No.

Date

分布式计算、大量通过网络连接的独立设备

紧密耦合

集群，tightly coupled的节点，相当于一台放大版系统

Cluster与Grid的区别：所有权是否唯一

Distributed Computing包含Cluster、Grid、Cloud、Edge.

provider mode（独立所有者）通过middleware（中间件）将多个不同的资源汇聚至一处完成一项共同的任务，它们的地点、硬件、系统等均可能不同.

user-node看到的是一个单一入口的useful pool（资源池）

VO: Virtual Organization，提供临时的temporary 共同目标shared goal、成员资格membership、角色roles、规划rules。它们共享使用权，但并不会合并.

本地优先：当本地有任务时，Grid将排队任务自动分配到其它资源上.

高 ↑ Grid Computing：Layer 1：Application：应用层. 解决用户实际问题

2：Middleware：中间件层. 协调user与provider的供求

3：Resources：资源层 暴露可用Capacity（能力）

4：Fabric：织构层 保障Capacity能被访问.

↓ 低

注：Middleware与VO不会凭空创造资源

同一个机构可能既为user也为provider.

No.

Date

例：500台电脑 under one administrate domain，为 cluster；

若想变为 Grid，则需通过 grid middleware 对外暴露并建立相同的 access policies（访问策略）

高通量 HTC：Grid　　高性能 HPC：Cluster

拥有大量独立作业，且作业间不通信　一个 tightly coupled 大任务、Low Latency

ClassAd：Job：　我需要什么？

Machine：我能给什么+策略。

例：Folding @ home.

例：500台电脑 under one administrate domain，其为 cluster；

若想为 Grid，则要通过 grid middleware 对外暴露并建立相同的 access policies（访问策略）

扫描全能王 创建

No.

Date

诸如

Grid访问本身就有风险，request access（访问请求）必须回答几个问题：

Authentication	认证	对方是谁？
Authorisation	授权	能允许做些什么？
Accounting & audit	审计	监视动作记录
Delegation	委托	权限与使用时间受限

注：A valid identity（合法身份） doesn't imply permission（有权限）, and permission doesn't remove the need for audit（审记）.

有身份（认证通过）不等于能授权，授权后不等于不被监视

Key pair：密钥对

Public key：公钥 .pub后缀 · Private key：私钥 无后缀

Shared openly：公开分享 remain secret 必须保密

Encrypts data for owners 加密数据 →（双向）Decrypts received data 解密数据

Verifies digital signatures 验证签名 ←Create digital signatures：创建签名

公钥不需保密，但公钥与身份保密绑定，故保护公钥也很重要

私钥通常是硬件密钥，禁止发送给他人，至则就失去了意义

扫描全能王 创建

No.

Date

PKI：一个 connecting（连接） public keys（公钥） to trusted identities（可信身份）.

帮助使用者确认公钥属于它声称的网站、组织等.

X.509 certification：证书，是一份公开的，被 CA（核验身份并签发证书. 确定身份与公钥绑定可信）数字签名的声明，绑定身份与公钥，说明 valid（有效期） 和 constrains（约束）.

identity（身份） + Public key（公钥） + Restriction（限制） —CA signature→ X.509 certificatior

Proxy 证书：short（短寿命） lifetime 并可进一步施加约束

Proxy 有自己的私钥签名，登录 server（服务器） 时用 Proxy certification 附带的公钥验证，故不需要用户私钥.

注：回答 Proxy 优点时一定要（说） 保护 private key 的重要性并说明分发 private key 的缺点

SSH 证书：让用户使用 CA 签名的公钥并证明自己拥有私钥.

以登录 SSH server，证书里写明 principal（身份） 和 lifetime（有效期）.

服务器文件对比：一工与一王让每次被使用都能追溯签发记录

id_grid：私钥	id_grid.pub：公钥.	id-grid-cert.pub：已签名的 SSH 证书
绝不离身.	用来交由 CA 签名.	登录时给 server 验证.

Trusted CA（可信CA） + valid certification（有效证书） + allowed principal（允许的身份） + privated-key proof（私钥持有证明） ⟹ Login accept（登录成功）.

扫描全能王 创建

No.

Date

Zero-trust decisions: 零信任，用户不会因为处于组织内部就被信任，仍然需要走验证流程。

CPU：最小化延迟 (Powerful core)

大缓存 Large caches + 复杂控制逻辑 complex control

让少数几条线程尽快跑完

GPU：最大化吞吐 throughput

海量线程隐藏延迟 many threads hide latency

擅长处理大量可反复利用的独立元素

注：短的串行链、heavily breaching、过短的并行操作均不适合GPU

CPU可以尽快跑完 串行分支多 启动开销比运行开销大

kernel

CPU: host (主机) → host memory ⟷ device memory ← GPU: device (设备)

数据搬运的开销大，故通常在GPU中经过多次运算后仅把最终结果传回CPU。也因此，把操作从GPU转到CPU会因为搬运数据这一步导致变慢

kernel：核函数，GPU对许多元素执行的同一函数，用大量独立线程提供并行工作，让GPU始终保持忙碌，由index(索引)挑出元素，元素间无通信

例：a[i] = update(a[i]) 适合

b[i] = b[i-1] + c[i] 不适合，元素间存在latency.

扫描全能王 创建

No.

Date

能不设计 kernel 就不设计，越往下设计越复杂且回报率低

Framework：框架，如：Pytorch，TensorFlow 等模型。

可代替程序员做：device-memory management 设备内存管理
host-device data movement. 主机-设备数据搬运
selection and launch of GPU kernel 选择、调用核函数

扫描全能王 创建